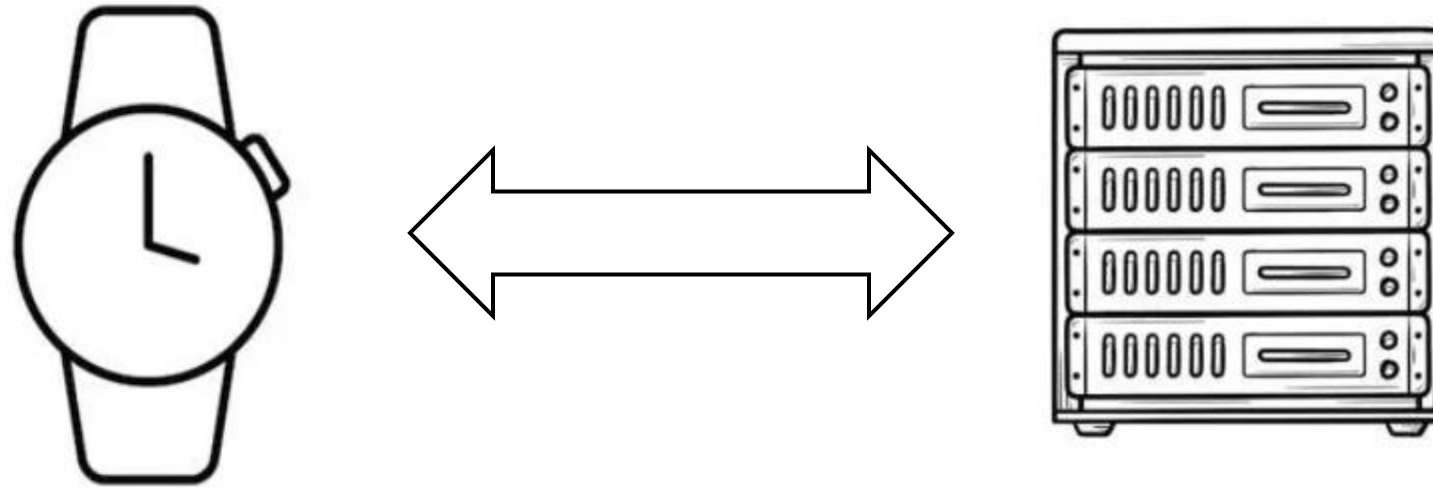


Réseaux: Projet 1: SRTP



Groupe 61

Jean-Louis Peffer 72232300

John Thiry 45942300

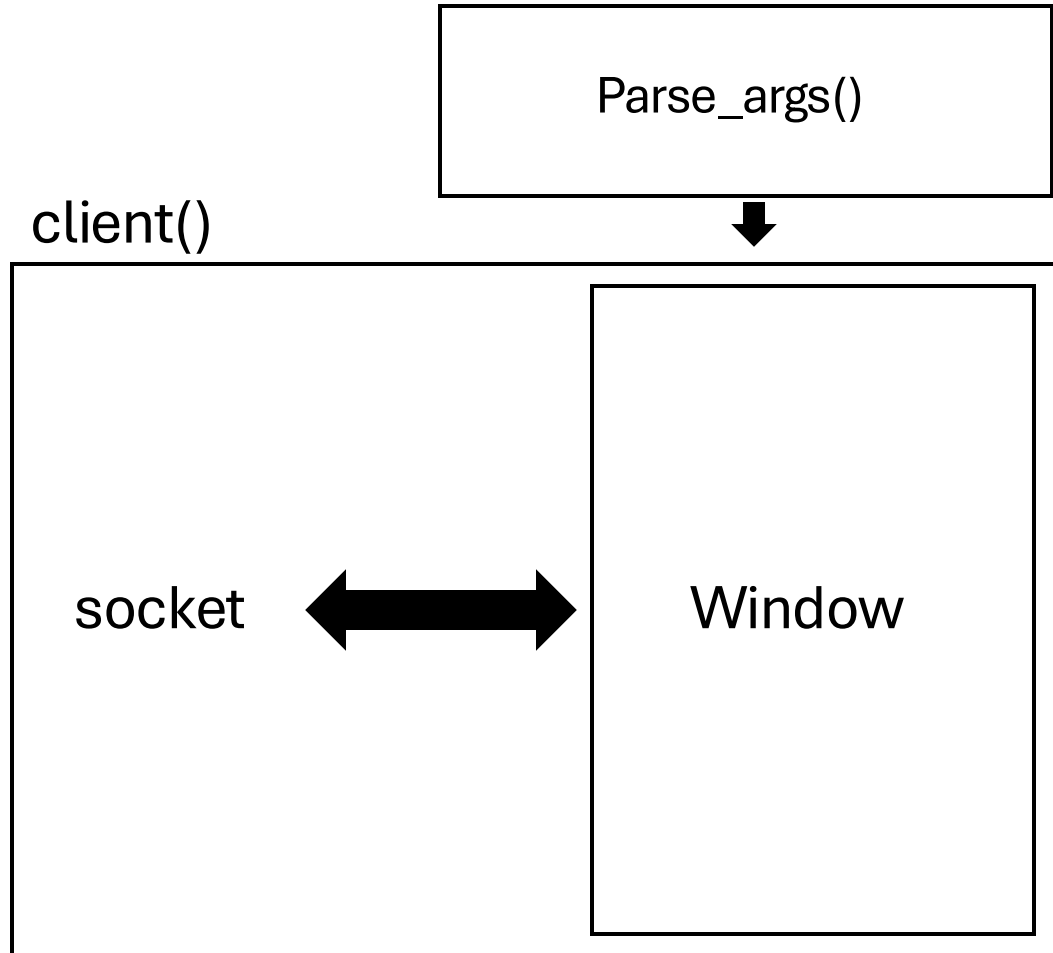
Eric Van Eetvelde 38962300



Structure du code

```
/
|-src /
|   | -client.py
|   | -code_pack.py
|   | -server.py
|-test/
|   | -Linksimulator-master/
|   | -black_box_test.py
|   | -structure_test.py
|-rapport.pdf
|-gitlog.stat
```

Client : implémentation



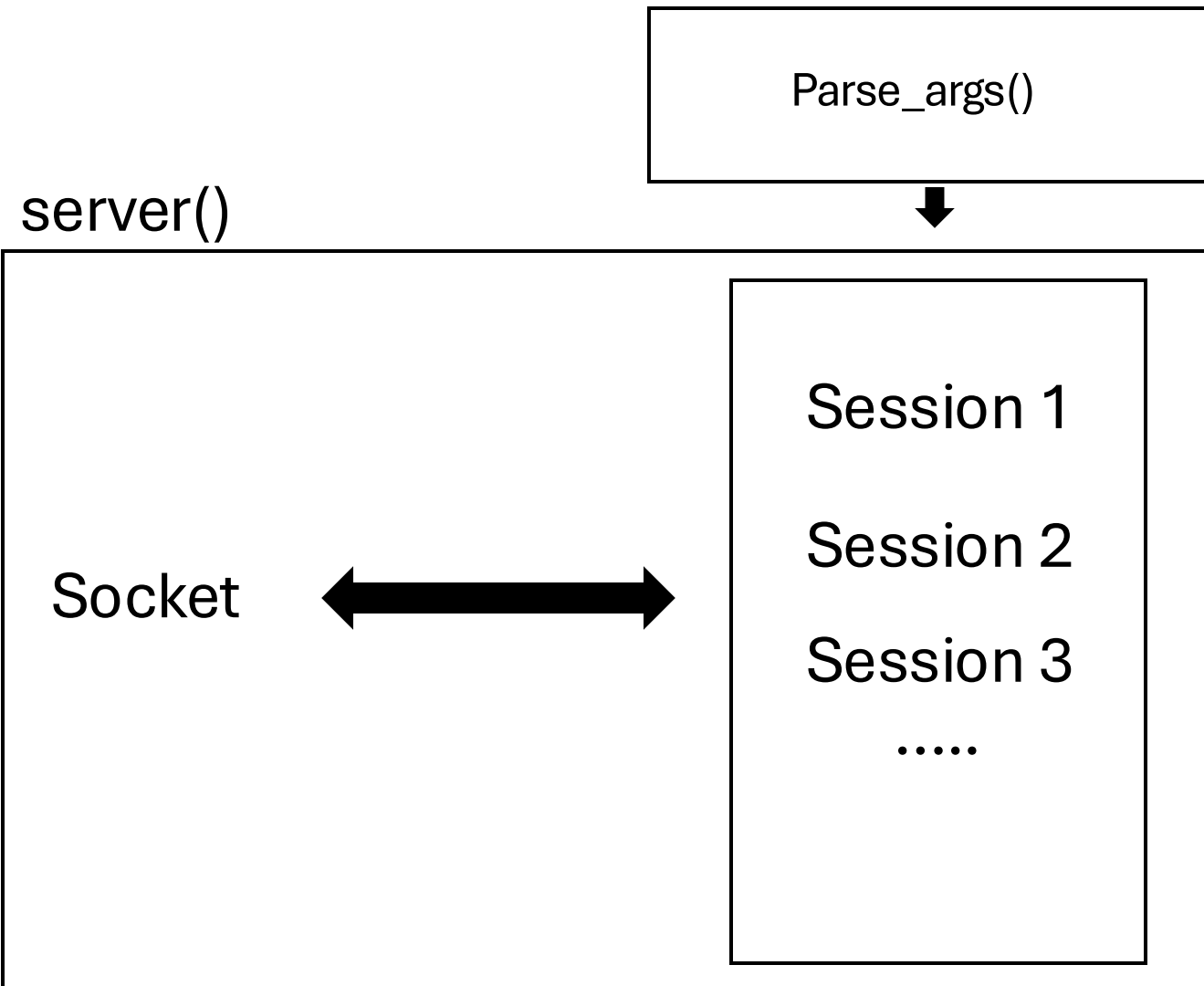
La fonction `parse_args()` sert à récupérer les arguments du programme.

La fonction `client()` tourne dans une boucle finissant uniquement quand la requête est totalement terminée.

L'objet `window` gère le réassemblage des paquets afin de les écrire dans l'ordre dans le fichier, et la fin du transfert est détectée via un paquet à payload vide.

L'objet `window` stocke les paquets dans une fenêtre ce qui permet de recevoir des paquets qui ne sont pas vraiment dans l'ordre

Serveur : implémentation



Le serveur peut gérer plusieurs clients en même temps grâce à un dictionnaire pouvant contenir plusieurs objets Session. Ces objets sont créés quand on reçoit une requête 'GET' et est associé à un client.

Le programme tourne dans une boucle principale avec un timeout de 1 seconde, quand il ne reçoit rien il vérifie les timeouts de chaque paquet de chaque session active et retransmet si nécessaire (avec backoff exponentiel);

Les sessions gèrent séparément les envois de paquets vers le client ainsi que le traitement des paquets. La session est détruite dès que tous les payloads sont acquittés.

Le programme serveur tourne indéfiniment sauf erreur à l'exécution.

Tests:

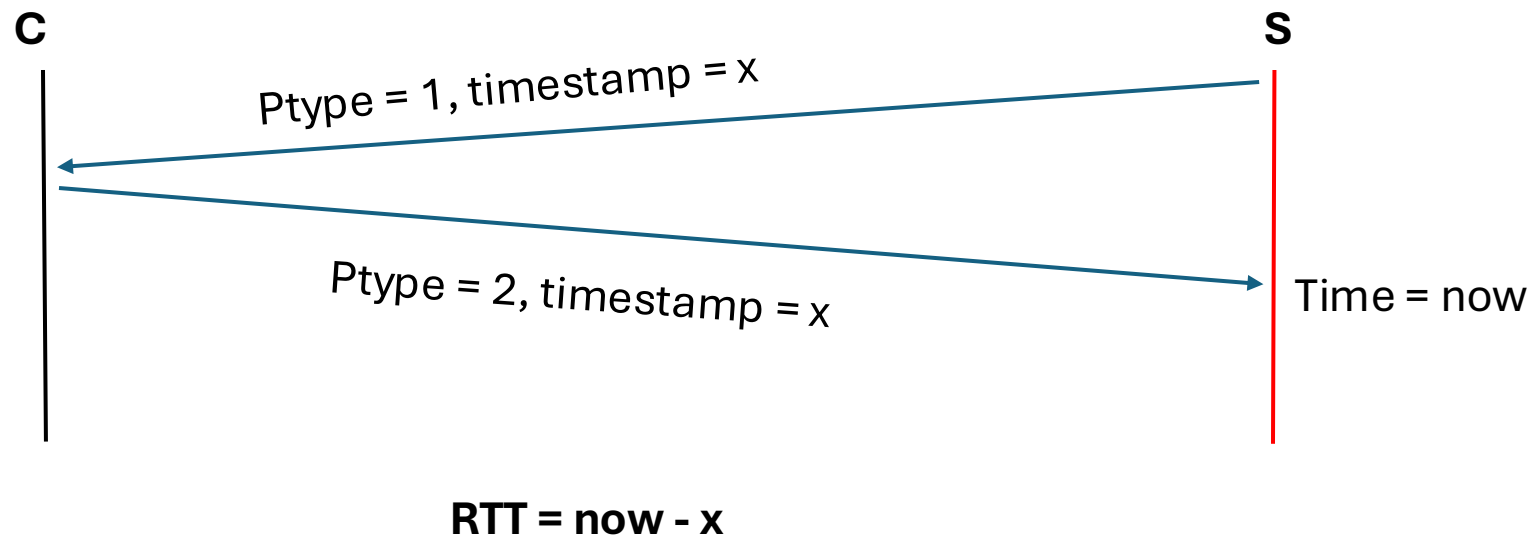
- Deux fichiers de tests.
- `Structure_tests.py` teste les fonctions internes des programmes.
- `Black_box_tests.py` teste le fonctionnement global des programmes ainsi que l'interaction entre ces derniers: le dialogue entre le serveur et le client, sous différentes conditions

Timestamp:

- Le champ timestamp contient le nombre de millisecondes écoulés depuis l'Epoch (1er janvier 1970).
- Il est utilisé par le client et le serveur afin de déterminer quand renvoyer le paquet de requête GET.
- Les paquets de type ACK envoyé par le client ont le même timestamp que le paquet de données reçu.
- Les timestamp sont utiles pour calculer le RTT (Round Trip Time) du côté serveur afin de savoir si il est nécessaire de renvoyer le paquet.

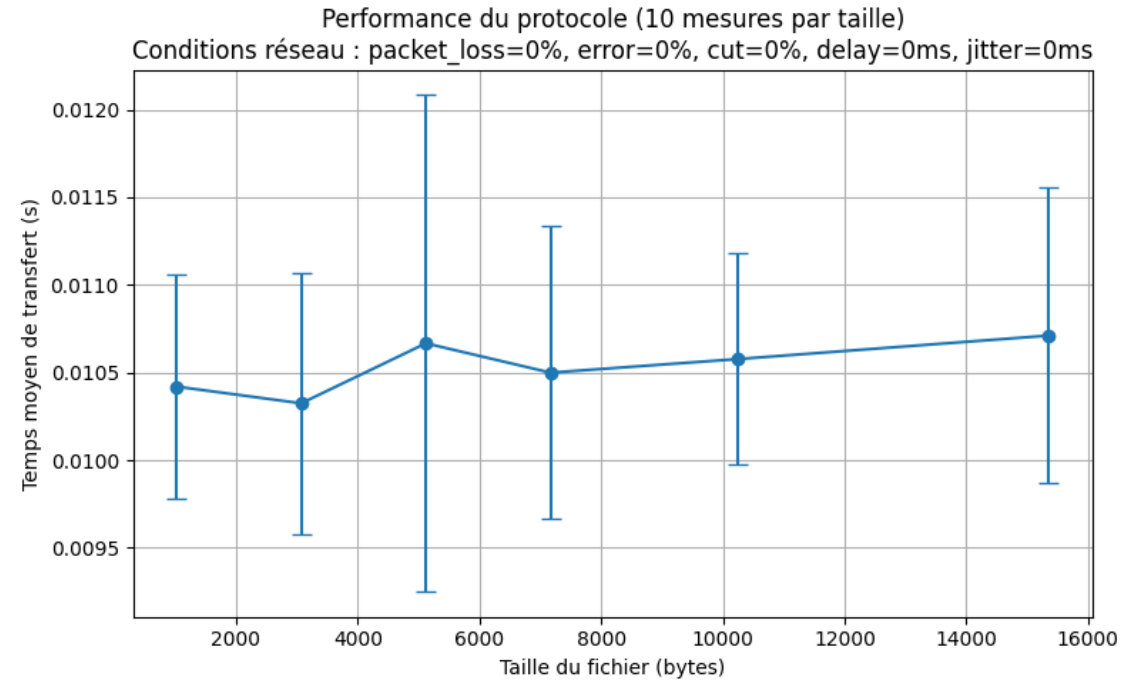
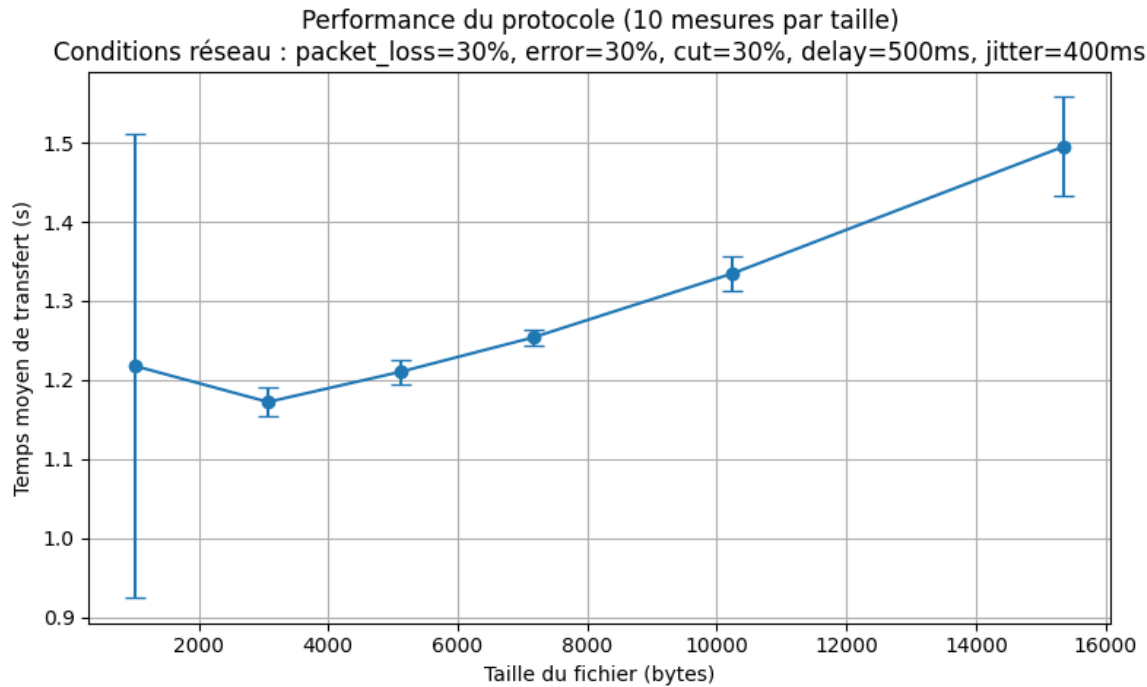
RTO:

- Le RTO est un champ défini pour chaque packet, il est utilisé uniquement par le serveur, il est initialisé à 1 seconde.
- Sur base du RTT celui-ci est mis à jour à la réception d'un "ack" chez le serveur.



- il est doublé à chaque nouvel envoi du paquet (exponential backoff), il est limité à 60 secondes.

Perfs et partie critique:



Voici deux graphes de performances: à gauche, en présence de défauts dans le réseau et à droite, dans un réseau parfait.

Serveur: prend du temps à vérifier les RTO

Client : Traitement des packets dans l'objet window

SACK: pas fait

- L'extension SACK n'a pas encore été implémenté